

Decision Tree Algorithm

Decision Tree :

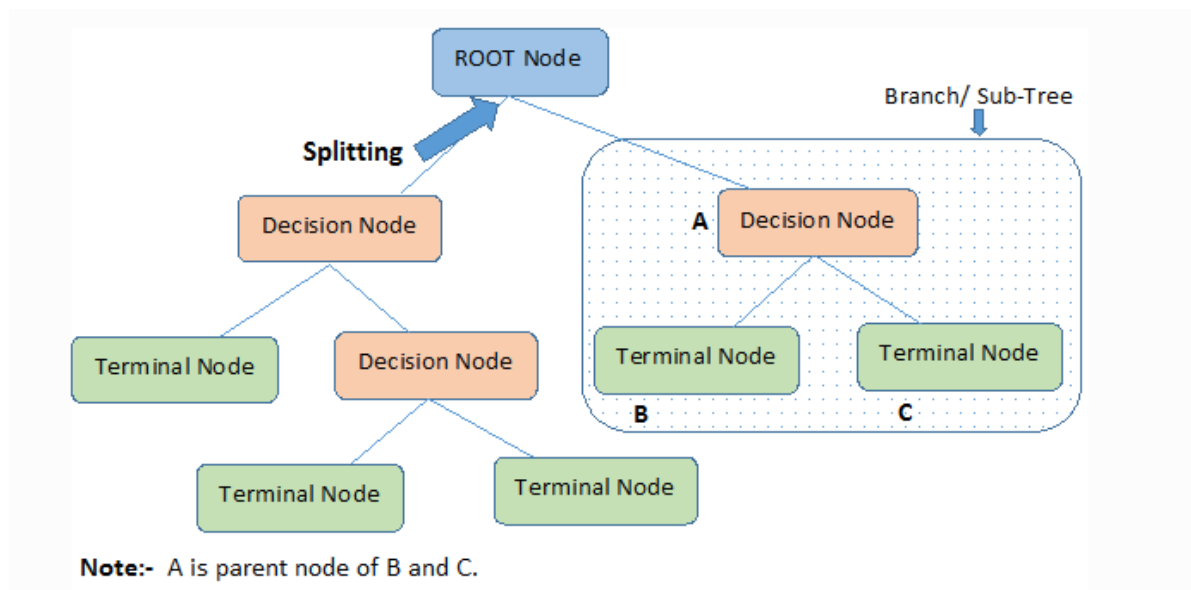
Decision Tree algorithm belongs to the family of **supervised learning algorithms**. Unlike other supervised learning algorithm the decision tree algorithm can be used for solving **regression and classification problems** too.

Types of Decision Trees :

1. **Categorical Variable Decision Tree:** Decision tree which has a categorical target variable then it is called a Categorical Variable decision tree.
2. **Continuous variable Decision Tree:** Decision tree has a continuous target variable then it is called Continuous Variable Decision tree.

Important Terminology related to Decision Trees:

1. **Root Node :** It represents the entire population or sample and this further gets divided into two or more homogeneous sets.
2. **Splitting:** it is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub node splits into further sub-nodes then it is called the decision node.
4. **Leaf/ Terminal Node:** Node that does not split is called leaf or sub-tree.
5. **Purning:** is nothing but cutting down some nodes to stop overfitting.
6. **Parent and Child Node:** A Node, which is divided into sub-nodes is called a parent node of sub-nodes whereas sub-nodes are called the child of a parent node.



The general steps to build a Decision tree are as follows:

- 1. Data Preparation :** Collecte and preprocess the data to be used in building the decision tree.
- 2. Attribute Selection :** identify the most import attributes of features in the data that will be used to build the decision tree. there are several techniques for attributes selection, such as **information gain, gain ratio and gini Index**
- 3. Tree building:** use an algorithm such as ID3,C4.5 or CART to construct the decision tree
The Involves recursively splitting the data into subset based on the selected attributes, until the tree is complete.
- 4. Tree Pruning:** Remove unnecessry branches or nodes from the decision tree to improve its accuracy and reduce overfiting there are several puring techniques such as reduced error purning and cost complexity purning

Attributes Selection:

if the dataset consists of N Attributes then deciding which attribute to place at the root or at different levels of the tree as internal nodes is complicated step. By just Randomly selecting any node to be the root can't sovl the issue. if we follow a random approach, it may give us bad result with low accuracy.

For solving this attribute selection problem researchers worked and devised some soluation. They suggested using some criteria like:

1. Entropy
2. information Gain
3. Gini Index

Entropy

In the context of decision tree, entropy is used as a measure of impurity or randomness in a set of data, and it help to determine the best split of dataset into subsets based on the value of a particular attribute. The goal is to minimize the entropy of each subset which menas that the resulting subsets are as pure as possible.

Entropy

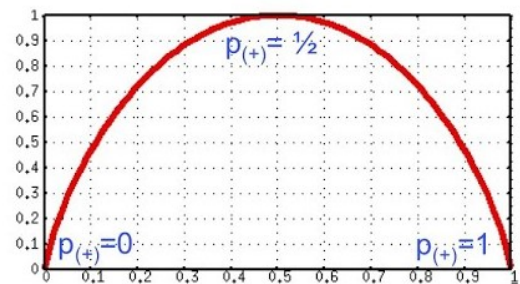
- Entropy: $H(S) = -p_{(+)} \log_2 p_{(+)} - p_{(-)} \log_2 p_{(-)}$ bits
 - S ... subset of training examples
 - $p_{(+)} / p_{(-)}$... % of positive / negative examples in S
- Interpretation: assume item X belongs to S
 - how many bits need to tell if X positive or negative

- impure (3 yes / 3 no):

$$H(S) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1 \text{ bits}$$

- pure set (4 yes / 0 no):

$$H(S) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0 \text{ bits}$$



Outlook	Temperature	Humidity	Wind	PlayGolf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rainy	Mild	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rainy	Mild	High	Strong	No

Yes $p = 9$

No $n = 5$

Total = 14

$$ENTROPY(S) = - (P_{\oplus}) \log_2 P_{\oplus} - P_{\ominus} \log_2 P_{\ominus}$$

$$S : [9+, 5-]$$

$$E = - \frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14}$$

$$= 0.940$$

Calculate using Python

```
In [1]: 1 import numpy as np
        2 from scipy.stats import entropy
        3
        4 base = 2
        5 pk = np.array([9/14, 5/14])
        6 h = entropy(pk, base=base)
        7 h
```

Out[1]: 0.940285958670631

Information Gain

Information gain (IG) measures how much "information" a features gives us about the class.it tell us how important a given attribute of the feature vectors is. Information gain (IG) is used to decide the ordering of attributes in the nodes of a decision tree.

$$Information\ Gain = E_{parent} - AvgE_{child}$$

Why we use the Information Gain ?

1. We want to determine **which attribute** in a given set of traning feature vectors is **most useful** for dicriminating between the classes to be learned.
2. **Information gain** tells us how important a given attribute of the features vectors is
3. we will use it to decide the ordering of attributes in the nodes of a decision tree.

Information steps:

The information gain is based on the decrease in entropy after a dataset is split on an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain (i.e., the most homogeneous branches).

step 1: Calculate entropy of the target.

$$\begin{aligned} Entropy(PlayGolf) &= Entropy(5,9) \\ &= Entropy(0.36, 0.64) \\ &= - (0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\ &= 0.94 \end{aligned}$$

step 2: The dataset is then split on the different attributes. The entropy for each branch is calculated. Then it is added proportionally, to get total entropy for the split. The resulting entropy is subtracted from the entropy before the split. The result is the Information Gain, or decrease in entropy.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

$$Gain(T, X) = Entropy(T) - Entropy(T, X)$$

$$G(\text{PlayGolf}, \text{Outlook}) = E(\text{PlayGolf}) - E(\text{PlayGolf}, \text{Outlook})$$

$$= 0.940 - 0.693 = 0.247$$

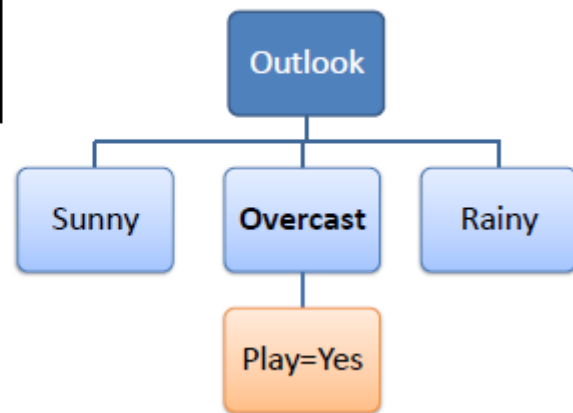
step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch.

		Play Golf	
		Yes	No
★ Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

Outlook		Temp	Humidity	Windy	Play Golf
Outlook	Sunny	Mild	High	FALSE	Yes
		Cool	Normal	FALSE	Yes
		Cool	Normal	TRUE	No
		Mild	Normal	FALSE	Yes
		Mild	High	TRUE	No
	Overcast	Hot	High	FALSE	Yes
		Cool	Normal	TRUE	Yes
		Mild	High	TRUE	Yes
		Hot	Normal	FALSE	Yes
	Rainy	Hot	High	FALSE	No
		Hot	High	TRUE	No
		Mild	High	FALSE	No
		Cool	Normal	FALSE	Yes
		Mild	Normal	TRUE	Yes

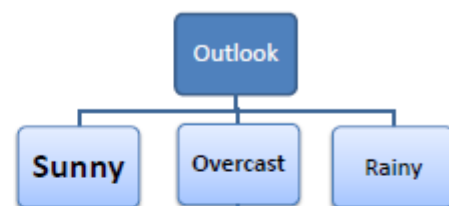
step 4: A branch with entropy of 0 is a leaf node.

Temp	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



step 5: A branch with entropy more than 0 needs further splitting.

Temp	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



Entropy VS Information Gain

Entropy and Gini impurity are both commonly used measures in decision tree algorithms to determine the best attribute for splitting the data.

Entropy is often used when the class distribution of the dataset is not imbalanced and the dataset has categorical features

Gini impurity is a good measure to use when the class distribution is imbalanced or there are continuous features in the dataset.

Gini Index

Gini Index is a measure used in decision tree to quantify the impurity of a dataset. it measures the probability of misclassifying a random sample from the dataset if it is randomly labeled according to the class distribution in the dataset.

The Gini index range from 0 to 1 where 0 indicates a completely pure dataset(i.e all samples belong to the same class) and 1 indicates a completely impure dataset(i.e the samples are evenly distributed across all classes.) in other words the lower the Gini index the better the split achieved by a decision tree algorithm.

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

Decision Tree Model

```
In [28]: 1 import pandas as pd
          2 import numpy as np
          3 import seaborn as sns
          4 import matplotlib.pyplot as plt
          5
```

```
In [3]: 1 data = pd.read_csv('train.csv')
```

```
In [4]: 1 data.columns
```

```
Out[4]: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
              'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
              dtype='object')
```

```
In [5]: 1 df = pd.read_csv('train.csv', usecols=['Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare'])
```

```
In [6]: 1 df.head()
```

```
Out[6]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare
0	0	3	male	22.0	1	0	7.2500
1	1	1	female	38.0	1	0	71.2833
2	1	3	female	26.0	0	0	7.9250
3	1	1	female	35.0	1	0	53.1000
4	0	3	male	35.0	0	0	8.0500

In [7]: 1 df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Survived    891 non-null    int64
1   Pclass      891 non-null    int64
2   Sex         891 non-null    object
3   Age         714 non-null    float64
4   SibSp       891 non-null    int64
5   Parch       891 non-null    int64
6   Fare        891 non-null    float64
dtypes: float64(2), int64(4), object(1)
memory usage: 48.9+ KB
```

Check the Duplicate Values

In [8]: 1 df.duplicated().sum()

Out[8]: 114

In [9]: 1 df.drop_duplicates(inplace=True)

In [10]: 1 df.duplicated().sum()

Out[10]: 0

In [11]: 1 df.info()

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 777 entries, 0 to 890
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Survived    777 non-null    int64
1   Pclass      777 non-null    int64
2   Sex         777 non-null    object
3   Age         674 non-null    float64
4   SibSp       777 non-null    int64
5   Parch       777 non-null    int64
6   Fare        777 non-null    float64
dtypes: float64(2), int64(4), object(1)
memory usage: 48.6+ KB
```

Handling missing value

```
In [12]: 1 df.isnull().sum()
```

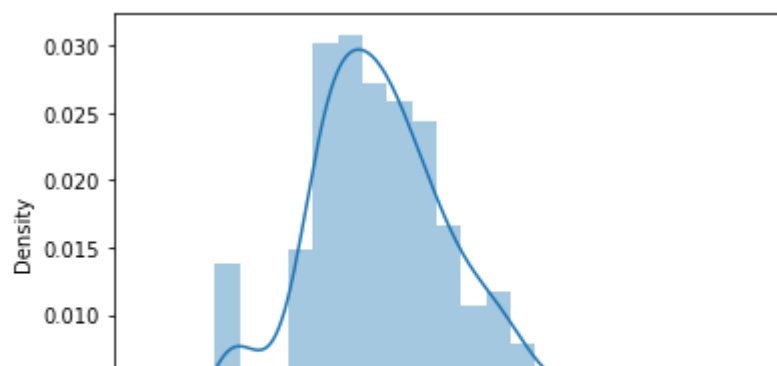
```
Out[12]: Survived      0  
Pclass      0  
Sex         0  
Age        103  
SibSp       0  
Parch       0  
Fare        0  
dtype: int64
```

We have some missing values for "Age" to fix it, we impute the missing value with mean() values.

```
In [13]: 1 sns.distplot(df['Age'])
```

```
C:\Users\Acer\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).  
warnings.warn(msg, FutureWarning)
```

```
Out[13]: <AxesSubplot:xlabel='Age', ylabel='Density'>
```



```
In [14]: 1 df['Age'].fillna(round(df['Age'].mean()),axis=0,inplace=True)
```

```
In [15]: 1 df['Age'].isnull().sum()
```

```
Out[15]: 0
```

Encoding Categorical features

To keep all the features in numeric format, we will encode the 'Sex', that means to transform the values from 'female' and 'male' to 0 and 1.

```
In [16]: 1 df['Sex'].value_counts()
```

```
Out[16]: male      486  
female    291  
Name: Sex, dtype: int64
```

```
In [17]: 1 from sklearn.preprocessing import LabelEncoder  
2 le = LabelEncoder()  
3 df['Sex'] = le.fit_transform(df['Sex'])
```

```
In [18]: 1 df.head()
```

```
Out[18]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare
0	0	3	1	22.0	1	0	7.2500
1	1	1	0	38.0	1	0	71.2833
2	1	3	0	26.0	0	0	7.9250
3	1	1	0	35.0	1	0	53.1000
4	0	3	1	35.0	0	0	8.0500

Splitting data set into train and test sets

```
In [19]: 1 x = df.iloc[:,1:] # Second column until the last column  
2 y = df.iloc[:,0] # First column (Survived) is our target  
3 from sklearn.model_selection import train_test_split  
4 #this function randomly split the data into train and test sets  
5 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=.3)  
6 #test_size=.3 means that our test set will be 30% of the train set.
```

Building the decision tree classifier

DecisionTreeClassifier() from sklearn is a good off the shelf machine learning model available to us. It has fit() and predict() methods. The fit() method is the “training” part of the modeling process. It finds the coefficients for the algorithm. Then, the predict() method will use the trained model to make predictions on a new set of data (test set).

```
In [20]: 1 from sklearn.tree import DecisionTreeClassifier  
2 dtree = DecisionTreeClassifier()  
3 dtree.fit(x_train, y_train) #train parameters: features and target  
4 pred = dtree.predict(x_test) #parameter: new data to predict
```

Accuracy

To determine how good our model is, we use another package from sklearn to give it a measure:

```
In [21]: 1 from sklearn.metrics import accuracy_score
2 accuracy_score(y_test, pred)
3 #parameters: targets to be predicted and predictions from new data used be

Out[21]: 0.7393162393162394
```

Accuracy : 0.739

it Means that we have 73% accuracy in our predications.

Visualizing the tree

We can plot the tree to see its root, branches, and nodes. We will use a set of new libraries to do so.

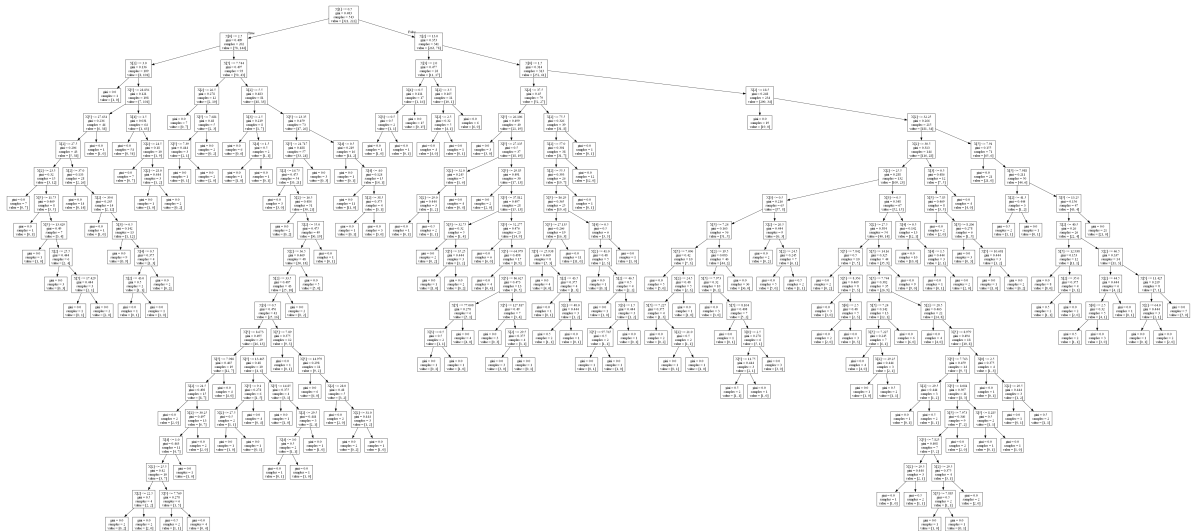
```
1 **StringIO():** creates an object (empty in this case) to receive a
   string buffer (the tree will be created first as a string before as an
   image) in DOT (graph description language) format.
2
3 **export_graphviz():** this function exports the tree in DOT format,
   generating a representation of the decision tree, which is written into
   the 'out_file'.
4
5 **graph_from_dot_data():** will use the DOT object to create the graph.
6
7 **Image():** uses raw data to return an image (png, jpeg, gif).
```

```

In [24]: 1 from sklearn.tree import export_graphviz
2 from six import StringIO
3 from IPython.display import Image
4 import pydotplus
5 dot_data = StringIO()
6 export_graphviz(dtrees, out_file=dot_data)
7 graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
8 graph.write_png('tree.png')
9 Image(graph.create_png())

```

Out[24]:



This tree seems pretty long. Let's change a couple of parameters to see if there is any effect on the accuracy and also to make the tree shorter.

Criterion: defines what function will be used to measure the quality of a split. The options are “gini” and “entropy”.

Max_depth: defines the maximum depth of the tree. If it's “none”, the tree will be as long as possible, when all the leaves are pure (risk of overfitting the model).

```

In [25]: 1 dtree = DecisionTreeClassifier(criterion='gini')
2 dtree.fit(x_train, y_train)
3 pred = dtree.predict(x_test)
4 print('Criterion=gini', accuracy_score(y_test, pred))
5 dtree = DecisionTreeClassifier(criterion='entropy')
6 dtree.fit(x_train, y_train)
7 pred = dtree.predict(x_test)
8 print('Criterion=entropy', accuracy_score(y_test, pred))

```

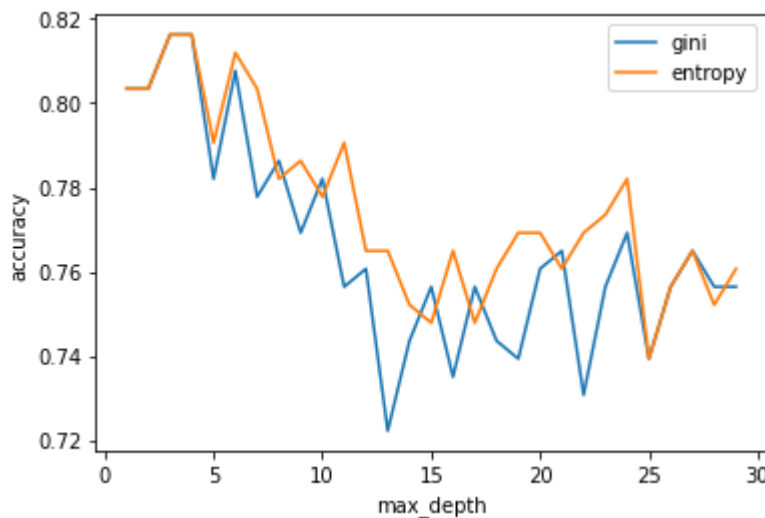
Criterion=gini 0.7393162393162394
 Criterion=entropy 0.7606837606837606

```

In [30]: 1 max_depth = []
2 acc_gini = []
3 acc_entropy = []
4 for i in range(1,30):
5     dtree = DecisionTreeClassifier(criterion='gini', max_depth=i)
6     dtree.fit(x_train, y_train)
7     pred = dtree.predict(x_test)
8     acc_gini.append(accuracy_score(y_test, pred))
9     #####
10    dtree = DecisionTreeClassifier(criterion='entropy', max_depth=i)
11    dtree.fit(x_train, y_train)
12    pred = dtree.predict(x_test)
13    acc_entropy.append(accuracy_score(y_test, pred))
14    #####
15    max_depth.append(i)
16    d = pd.DataFrame({'acc_gini':pd.Series(acc_gini),
17                     'acc_entropy':pd.Series(acc_entropy),
18                     'max_depth':pd.Series(max_depth)})
19    # visualizing changes in parameters
20    plt.plot('max_depth', 'acc_gini', data=d, label='gini')
21    plt.plot('max_depth', 'acc_entropy', data=d, label='entropy')
22    plt.xlabel('max_depth')
23    plt.ylabel('accuracy')
24    plt.legend()

```

Out[30]: <matplotlib.legend.Legend at 0x1c624520688>



```

In [32]: 1 dtree = DecisionTreeClassifier(criterion='entropy', max_depth=7)
2 dtree.fit(x_train, y_train)
3 pred = dtree.predict(x_test)
4 accuracy_score(y_test, pred)
5

```

Out[32]: 0.8205128205128205

We went from 0.73 of accuracy to 0.82 just by changing the fucation to create the branches and limiting the tree to avoid overfitting. Each case is different and it's a good practice to test different parameters to find the best for your scenario.

Advantages and Disadvantages of using decision trees :

Advantages:

1. **Easy to understand:** Decision trees are intuitive and easy to interpret, making them an ideal algorithm for beginners.
2. **Handles both categorical and numerical data:** Decision trees can handle both categorical and numerical data, unlike some other algorithms that are limited to one or the other.
3. **Handles missing values:** Decision trees can handle missing values in the data without requiring imputation.
4. **Non-parametric:** Decision trees do not make assumptions about the distribution of the data, which means they can work well with both linear and non-linear data.
5. **Feature importance:** Decision trees can identify which features are most important in predicting the target variable, allowing for easy feature selection.

Disadvantages:

1. **Overfitting:** Decision trees are prone to overfitting, especially if the tree is deep or the data is noisy.
2. **Instability:** Small changes in the data can cause large changes in the decision tree, making them unstable.
3. **Bias:** Decision trees can be biased towards features with more levels, which can lead to incorrect classifications.
4. **Limited to axis-parallel splits:** Decision trees are limited to creating splits that are orthogonal to the feature axes, which may not be optimal for all data sets.
5. **Computational complexity:** Decision trees can become computationally expensive as the number of features and data points increases, making them less suitable for very large data sets.